

P1175R0

a simple and practical optional reference for C++

Published Proposal, 2018-10-06

Author:

[JeanHeyd Meneide](#)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/ThePhD/future_cxx/blob/master/papers/source/d1175.bs

Current:

https://rawgit.com/ThePhD/future_cxx/papers/d1175.html

Reply To:

[JeanHeyd Meneide](#), [@thephantomderp](#)

Abstract

optional has seen much contention over its semantics and other points related to Regularity and Consistency, but not being able to answer these questions has delayed important advances in expected, variant, optional and other vocabulary types. This proposal demonstrates a safe and simple subset of optional that can gain consensus and still leaves the door open for when questions of consistency and regularity can be answered with confidence.

Table of Contents

1 Revision History

1.1 Revision 0

2 Overview

2.1 The Great Big Table of Behaviors

3 Motivation

4 Design Considerations

4.1 The Solution

5 Implementation Experience

6 Wording

6.1 Intent

6.2 Feature Test Macro

6.3 Proposed Wording

7 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release.

§ 2. Overview

§ 2.1. The Great Big Table of Behaviors

Below is a succinct synopsis of the options presented in this paper and their comparison with known solutions and alternative implementations. It does not include the totality of the optional API surface, but has the most exemplary pieces. A key for the symbols:

✓ - Succeeds

⊘ - Compile-Time Error

✗ - Runtime Error

? - Implementation Inconsistency (between engaged/unengaged states, runtime behaviors, etc.)

optional behaviors			
Operation	T	std::reference_wrapper<T>	<i>Proposed:</i> <u>T& conservative</u>
exemplary implementation(s)	✓ std::optional nonstd::optional llvm::Optional folly::Optional core::Optional	✓ std::optional nonstd::optional llvm::Optional folly::Optional core::Optional	✓ std::experimental::optional sol::optional
<code>optional(const optional&)</code>	✓ copy constructs T (disengaged: nothing)	✓ binds reference (disengaged: nothing)	✓ binds reference (disengaged: nothing)
<code>optional(optional&&)</code>	✓ move constructs T (disengaged: nothing)	✓ binds reference (disengaged: nothing)	✓ binds reference (disengaged: nothing)
<code>optional(T&)</code>	✓ (copy) constructs T	✓ binds reference	✓ binds reference
<code>optional(T&&)</code>	✓ (move) constructs T	⊘ compile-time error	⊘ compile-time error
<code>operator=(T&)</code> <i>engaged</i>	✓ overwrites T	✓ rebinds data	⊘ compile-time error
<code>operator=(T&)</code> <i>disengaged</i>	✓ overwrites data	✓ rebinds data (overwrites reference wrapper)	⊘ compile-time error
<code>operator=(T&&)</code> <i>engaged</i>	✓ move-assigns T	⊘ compile-time error	⊘ compile-time error
<code>operator=(T&&)</code> <i>disengaged</i>	✓ constructs T	⊘ compile-time error	⊘ compile-time error
<code>operator=(T&)</code> <i>engaged</i>	✓ overwrites T	⊘ compile-time error	⊘ compile-time error

operator= (optional <T>&) <i>disengaged</i>	✓ overwrites data	✓ overwrites data	✗ compile-time error
operator= (optional <T>&&) <i>engaged;</i> <i>arg engaged</i>	✓ move assign T	✓ rebind data	✓ rebind data
operator= (optional <T>&&) <i>disengaged;</i> <i>arg engaged</i>	✓ move construct T	✓ rebind data	✓ rebind data
operator= (optional <T>&&) <i>engaged;</i> <i>arg disengaged</i>	✓ disengage T	✓ disengage T	✓ disengage T
operator= (optional <T>&&) <i>disengaged;</i> <i>arg disengaged</i>	✓ nothing	✓ nothing	✓ nothing
*my_op = value <i>engaged</i>	✓ copy assigns T	✓ copy assigns T	✓ copy assigns T
*my_op = value <i>disengaged</i>	✗ runtime error	✗ runtime error	✗ runtime error
*my_op = std::move(value) <i>engaged</i>	✓ move assigns T	✓ move assigns T	✓ move assigns T
*my_op = std::move(value) <i>disengaged</i>	✗ runtime error	✗ runtime error	✗ runtime error
(*my_op). some_member() <i>engaged</i>	✓ calls some_member()	✗ compile-time error	✓ calls some_member()
(*my_op). some_member() <i>disengaged</i>	✗ runtime error	✗ runtime error	✗ runtime error
operator==(const optional&)	✓ compares values if both engaged, returns true if both disengaged, returns false otherwise	✓ compares values if both engaged, returns true if both disengaged, returns false otherwise	✗ compile-time error

<code>operator< (const optional&)</code>	✓ compares values if both engaged, returns true if both disengaged, returns false otherwise	✓ compares values if both engaged, returns true if both disengaged, returns false otherwise	✗ compile-time error
<code>operator== (const T&) engaged</code>	✓ compares values	✓ compares values	✗ compile-time error
<code>operator< (const T&) engaged</code>	✓ compares values	✓ compares values	✗ compile-time error
<code>operator== (const T&) disengaged</code>	✓ returns false	✓ returns false	✗ compile-time error
<code>operator< (const T&) disengaged</code>	✓ returns false	✓ returns false	✗ compile-time error

§ 3. Motivation

Originally, `std::optional<T>` -- where `T` denotes the name of a type -- contained a specialization to work with regular references, `std::optional<T&>`. When some of the semantics for references were called into question with respect to assign-through semantics (assign into the value or rebind the optional) and how comparisons would be performed, the debate stopped early and no full consensus was reached. Rather than remove just the operator or modify comparison operators, the entirety of `std::optional<T&>` was removed entirely.

This left many codebases in an interesting limbo: previous implementations and external implementations handled references without a problem. Transitioning to pointers created both a problem of unclear API (pointers are an exceedingly overloaded construct used for way too many things) and had serious implications for individuals who wanted to use temporaries as part of their function calls.

As Library Evolution Working Group Chair Titus Winters has frequently stated and demonstrated, having multiple vocabulary types inhibits growth of the C++ ecosystem and fragments libraries and their developers. This comes at an especially high cost for `optional`, `variant`, `any`, `expected` and more. There are at least 6 different `optionals` in the wild with very slightly differing semantics, a handful more `variants`, a few `expected` types, and more (not including the ones from `boost::`). Of note is that many optionals have been created and are being nurtured to this day without the need to take care of legacy code, which greatly inhibits interoperability between code bases and general sharing.

§ 4. Design Considerations

This solution is the simplest cross-section that enables behavior without encroaching upon a future where the to be posed in the yet-to-be-released p1129r0 will reach an answer to move the C++ community forward. Care has been taken to only approach the most useful subsection, while keeping everything else deleted. This will enable developers to use optional for the 80% use cases, while users handle the 20% use case of rebinding, assigning through, or comparing values / location by choosing much more explicit syntax that will not be deprecated.

§ 4.1. The Solution

This baseline version is the version that has seen adoption from hundreds of companies and users: an optional where `operator=` is not allowed, comparison operators are nuked, rebinding is done with an explicit wrapping of `my_op = std::optional<T&>( my_lvalue )` and assign-through is performed using `*my_op = my_l_or_rvalue`. This keeps `std::optional` as a delay-constructed type, allows usage in all of the places a programmer might want to put it trivially, allows it to be used as a parameter, and allows it to be used as a return type.

It forces the user to choose assign-through by explicitly dereferencing the optional as in `*my_op = some_value`, and forces rebind by making the user specify `optional<T&&>( some_lvalue )`. It is safe, but penalizes the user for this safety with verbosity (and, arguably, disappointment). It also prevents users of `boost::optional`, `tl::optional`, `ts::optional_ref` and others from migrating painlessly to the standard version, but still allows many of the high-priority uses of such classes with references to transition to using the standard library version.

Another notable feature of adding optional references and using `const T&` is the ability to transition codebases that use temporary values (r-values) passed to functions, this solution will work for individuals without requiring a full rewrite of the code. For example, the function `void old_foo( int arg, const options& opts );` can be transitioned to `void old_foo( int arg, optional<const options&> opts );` and work for both l-values and r-values passed to the type. This is safe thanks to C++'s lifetime rules around temporary objects, when they bind to references, and when they are lifetime extended; see [class.temporary]/6 for applicable lifetime extension clauses of temporaries, including temporaries that bind to stored references.

§ 5. Implementation Experience

This "simple", baseline version is featured in [akrzemi/optional](#), [\[sol2\]](#), and the "portable" version of [\[boost-optional\]](#) (following Boost's advice to avoid the use of the assignment operator in select cases for compilers with degenerate behavior). It is the least offensive, tasteless, hazard-proof, odorless, non-toxic, biodegradable, organic, and politically correct choice™; it can also be expanded upon at a later date.

This specific version has seen experience for about 8+ years. It is known to be safe and easy to use and covers a good portion of user's use cases without having to invoke the problem of figuring out assign-through or rebind semantics.

The comparison operators do not exist, which make it a subset of `boost` and other optional implementations. Additional work can be done later once the Committee and its constituents .

§ 6. Wording

All wording is relative to [\[n4762\]](#).

§ 6.1. Intent

The intent of this proposal is to provide a l-value reference optional. The comparison and equality operators will not be provided. The assignment operator from an l-value reference or from an l-value reference of its base will not be provided. The copy-assignment from two optionals will rebind the optional.

Comparison to `nullopt` with equality will provided.

§ 6.2. Feature Test Macro

The proposed feature test macro is `__cpp_lib_optional_ref`.

§ 6.3. Proposed Wording

Append to §16.3.1 General `[support.limits.general]`'s **Table 35** one additional entry:

Macro name	Value
<code>__cpp_lib_optional_ref</code>	201811L

Add additional class template specialization to §19.6.2 Header `<optional>` synopsis `[optional.syn]`:

```
// 19.6.3, class template optional
template<class T>
class optional;
```

```
// 19.6.4, class template optional for lvalue reference types
template<class T>
class optional<T&>;
```

Insert §19.6.4 `[optional.lref]` after §19.6.3 Class template `optional` `[optional.mod]`:

19.6.4 Class template optional for lvalue reference types [optional.lref]

```
namespace std {  
  
    template <class T>  
    class optional<T&> {  
    public:  
        typedef T& value_type;  
  
        // 19.6.4.1, construction/destruction  
        constexpr optional() noexcept;  
        constexpr optional(nullopt_t) noexcept;  
        constexpr optional(T&) noexcept;  
        optional(T&&) = delete;  
        constexpr optional(const optional&) noexcept;  
        template <class U> optional(const optional<U&>&) noexcept;  
        ~optional() = default;  
  
        // 19.6.4.2, mutation  
        constexpr optional& operator=(nullopt_t) noexcept;  
        optional& operator=(optional&&) = delete;  
        optional& operator=(const optional&) = delete;  
  
        // 19.6.4.3, observers  
        constexpr T* operator->() const;  
        constexpr T& operator*() const;  
        constexpr explicit operator bool() const noexcept;  
        template<class U> constexpr T value_or(U&&) const&;  
  
        -  
        // 19.6.4.4, modifiers  
        void reset() noexcept;  
  
    private:  
        T* ref; // exposition only  
    };  
  
} // namespace std
```

¹ Engaged instances of `optional<T>` where `T` is of lvalue reference type, refer to objects of type `std::remove_reference_t<T>`, but their life-time is not connected to the life-time of the referred to object. Destroying or disengaging the optional object does not affect the state of the referred to object.

² Member `ref` is provided for exposition only. Implementations need not provide this member. If `ref == nullptr`, optional object is disengaged; otherwise `ref` points to a valid object.

19.6.4.1 Construction and destruction [optional.lref.ctor]


```
constexpr optional<T&>::optional() noexcept;
constexpr optional<T&>::optional(nullopt_t) noexcept;
```

¹ Effects: Constructs a disengaged optional object by initializing ref with `nullptr`.

² Ensures: `bool(*this) == false`.

³ Remarks: For every object type `T` these constructors shall be constexpr constructors.

```
optional<T&>::optional(T& v) noexcept;
```

⁴ Effects: Constructs an engaged optional object by initializing ref with `addressof(v)`.

⁵ Ensures: `bool(*this) == true && addressof>(*this) == addressof(v)`.

```
optional<T&>::optional(const optional& rhs) noexcept;
template <class U> optional<T&>::optional(const optional<U&>& rhs) noexcept;
```

⁶ Constraints: `is_base_of<T, U>::value == true`, and `is_convertible<U&, T&>::value` is true.

⁷ Effects: If rhs is disengaged, initializes ref with `nullptr`; otherwise, constructs an engaged object by initializing ref with `addressof(*rhs)`.

```
optional<T&>::~~optional() = default;
```

⁹ Effects: No effect. This destructor shall be a trivial destructor.

19.6.4.2 Mutation [`optional.lref.mutate`]

```
optional<T&>& optional<T&>::operator=(nullopt_t) noexcept;
```

¹ Effects: Assigns ref with a value of `nullptr`. If ref was non-null initially, the object it referred to is unaffected.

² Returns: `*this`.

³ Ensures: `bool(*this) == false`.

19.6.4.3 Observers [`optional.lref.observe`]

```
T* optional<T&>::operator->() const;
```

¹ Requires: `bool(*this) == true`.

² Returns: `ref`.

³ Throws: nothing.

```
T& optional<T&>::operator*() const;
```

⁴ Requires: `bool(*this) == true`.

⁵ Returns: `*ref`.

⁶ Throws: nothing.

```
explicit optional<T&>::operator bool() noexcept;
```

⁷ Returns: `ref != nullptr`.

```
template <class U> T& optional<T&>::value_or(U&& u) const;
```

⁸ Returns: `.value()` when `bool(*this)` is true, otherwise `std::forward<U>(u)`.

19.6.4.4 Observers [optional.lref.modifiers]

```
template <class U> T& optional<T&>::value_or(U&& u) const;
```

¹² Returns: `.value()` when `bool(*this)` is true, otherwise `std::forward<U>(u)`.

Modify §19.6.6 Relational operators [optional.relops] to include the following top-level clause:

¹ None of the comparisons in this subsection participate in overload resolutions if `T` or `U` in `optional<T>` or `optional<U>` are an l-value reference.

Modify §19.6.8 Comparison with `T` [optional.comp_with_t] to include the following top-level clause:

¹ None of the comparisons in this subsection participate in overload resolutions if `T` or `U` in `optional<T>` or `optional<U>` are an l-value reference.

§ 7. Acknowledgements

Thank you to sol2 users for encouraging me to fix this in the standard. Thank you to Lisa Lippincott for encouraging me to make this and one other proposal after seeing my C++Now 2018 presentation. Thank you to Matt Calabrese, R. Martinho Fernandes and Michał Dominiak for the advice on how to write and handle a paper of this magnitude.

Thank you to Tim Song and Walter Brown for reviewing one of my papers, and thus allowing me to improve all of them.

§ References

§ Informative References

[AKRZEMI-OPTIONAL]

Andrzej Krzemiński. [Optional \(nullable\) objects for C++14](https://github.com/akrzemi1/Optional). April 23rd, 2018. URL: <https://github.com/akrzemi1/Optional>

[BOOST-OPTIONAL]

Fernando Luis Cacciola Carballal; Andrzej Krzemiński. [Boost.Optional](https://www.boost.org/doc/libs/1_67_0/libs/optional/doc/html/index.html). July 24th, 2018. URL: https://www.boost.org/doc/libs/1_67_0/libs/optional/doc/html/index.html

[FOLLY-OPTIONAL]

Facebook. [folly/Optional](https://github.com/facebook/folly). August 11th, 2018. URL: <https://github.com/facebook/folly>

[LLVM-OPTIONAL]

LLVM Developer Group. [Optional.h](http://llvm.org/doxygen/Optional_8h_source.html). July 4th, 2018. URL: http://llvm.org/doxygen/Optional_8h_source.html

[MARTINMOENE-OPTIONAL]

Martin Moene. [Optional Lite](https://github.com/martinmoene/optional-lite). June 21st, 2018. URL: <https://github.com/martinmoene/optional-lite>

[MNMLSTC-OPTIONAL]

Isabella Muerte. [core::optional](https://mnmlstc.github.io/core/optional.html). February 26th, 2018. URL: <https://mnmlstc.github.io/core/optional.html>

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [n4762 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4750.pdf). May 7th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4750.pdf>

[SOL2]

ThePhD. [sol2: C++ <-> Lua Binding Framework](https://github.com/ThePhD/sol2). July 3rd, 2018. URL: <https://github.com/ThePhD/sol2>